Technische Hochschule Ingolstadt

---

# Bachelor Thesis

Faculty of computer science

## Development of a Frontend Application for Real-Time and 24-Hour Monitoring and Recording of Electrocardiographic Data with Communication via Bluetooth Low Energy


First- and Lastname : **Eric Horacsek**



Registered on : 25.11.2024

Submitted on : xx.yy.zzzz

First examiner : Prof. Dr.-Ing. Georg Passig

Second examiner : Prof. Dr. Ulrich Margull

# Affidavit

I hereby declare that I have written this thesis independently, have not used any sources or aids other than those stated, have not submitted it elsewhere for examination purposes and have expressly identified all quotations from the sources used, either verbatim or in spirit. I have marked verbatim and indirect quotations as such.

Ingolstadt, _____

Firstname Lastname

## Acknowledgements

First and foremost, I would like to express my deepest gratitude to Prof. Dr. Passig for his outstanding supervision and mentoring throughout the course of this thesis. His invaluable guidance, insightful feedback, and unwavering support have been monumental.

A heartfelt thank you to Stefan Lindner and Sebastian Hardt, whose camaraderie made the long hours of studying not only bearable but also enjoyable.

I am also grateful to the TSV Hustler's for their continuous support and for always believing in me. Their enthusiasm and encouragement have helped me stay focused and driven.

A very special thank you to Sarah Hörmann, my girlfriend, for her patience, understanding, and unwavering support during the countless hours I was absent.

Eric Horacsek
Ingolstadt, Deutschland
November xx 2024

## Abstract

The human heart is the central organ of the circulatory system and plays a vital role in supplying the body with oxygen and nutrients. Rhythmic contractions pump blood through the body, laying the foundation for all vital functions. An essential part of medical diagnostics combined with heart function and health is the electrocardiogram (ECG). The ECG measures the electrical activity of the heart and provides valuable information about the heart rhythm aswell as possible irregularities or diseases. As technology has advanced, the way in which ECG data can be collected and monitored has also evolved significantly. Mobile health monitoring in particular now makes it possible to record ECG data not only at certain points of time, but also in real time and over longer periods of time. Wireless transmission via Bluetooth Low Energy (BLE) enables reliable and energy-efficient transfer of data to mobile devices. This opens up new possibilities for applications offering real-time and long term monitoring enabling users access to relevant information and graphical processing and alarm functions. This thesis explains the conception and implementation of an Android frontend application, which realises continuous and mobile ECG monitoring. The main programming language used for this project, is kotlin with Jetpack Compose as the Framework. Moreover this dissertation is particulary focused on implemententing the communication and data transmission via BLE aswell as realising an 24-hour-mode, which might be used for 24-hour data recognition. It also provides an broad insight into the given requirements, designprincipals, and technical challenges which accured during the implementation. Furthermore this theseis offers a glimplse into the potential for modern healthsuveillance.

## Confidentialityclause

Optional.

Ingolstadt, _____
(Date)

(Signature)
Firstname Lastname

# Glossary

**Eric Horacsek** Absolute operation.

# Acronyms

**D** Dimensions.

**RGB** Red Green Blue channels.

# List of figures

# List of tables

# List of contents

# 1 Introduction

This chapter will give a small introduction on what the thesis and the implemented application is about. This covers an explanation for the application's motivation and the environment the application will be used in.

## 1.1 Motivation

The aim of this work is to create a new version of an Android App once developed by fellow students from the faculty of Electrical and Information Technology. It's core functionallity is to receive and display ECG data in real time, via BLE. With the completion of this work, the new version will be able to receive and display ECG data in real time, via BLE. In addition it will be capable of an 24-hour mode which records ECG data for up to 24 hours, so the collected data can be analyzed later.

## 1.2 Scope and Framework

This paper only deals with the app development process of the new version. Therefore, any implementation of the previous work done by the students is not going to be discussed. The hardware implementation is also not an element of this work. Generally speaking, this paper covers all topics that are essential for the development of the new app version. Specifically, technology stack of choice, IDE choice desicion, communication with hardware via Bluetooth low energy, structure of the user interface, real-time data monitoring, implementing the 24 hour mode. In addition, the outlook explains procedures for the implementation of future features that could not be realized within the limits of this paper.

## 1.3 Procedure

The second chapter, *Technical Fundamentals*, explores the Android architecture, with a particular focus on the Java Framework API. Giving valuable insights into the core functionality of the operating system, laying the groundwork for a better understanding of the implementation details presented later. Chapter three delves into the functional requirements, systematically breaking them down into distinct sub-requirements. These requirements are revisited in chapter four, where they are translated into actual code, accompanied by relevant code snippets to illustrate their implementation.

# 2 Technical Fundamentals

## 2.1 Basics of Android architecture

To program a platform-specific application, it can be important to understand the basic architecture of the platform, especially when dealing with taks regarding memory or Bluetooth. This section will give a brief insight into the structure of the Android construction. Tasks discussed later in this thesis may reference this chapter to convey a better understanding.

The core of the Android Platform is a Linux based kernel. On top of Android's Kernel are four more stack layers, such as the Hardware Abstraction Layer, Native Libraries/Android Runtime, Java API Framework and the uppermost layer, the System App layer. (also see: Figure 1). Going into detail for each stack layer is not necessary for this work, consequently only essential layers like the Java API Framework will be considered in more depth. A short introduction into each layer is given by table 1 and figure 1 .

**Table 1:** Layers explanation

| **Layer** | **Functionality** |
|---|---|
| System App's (Application Layer) | This layer is a collection of preinstalled applications. Apps contained in this collection, have the same status at system level as third-party apps(system settings is the only exception). Giving the user freedom for design. |
| Java API Framework (Framework Layer) | Enables acces to all features of the Android OS, such as resource manager and view system. |
| Native Libraries | System components such as ART and HAL are written in C/C++, those functionalities can be used via API's |
| Android Runtime (ART) | Every running application is an instance of ART. ART operates on the same layer as Native Libraries |
| Hardware Abstraction Layer (HAL) | Provides interfaces to the Java API Framework, hence addresing hardware components like Camera or Bluetooth module is possible. |
| Linux Kernel | Foundation of the OS. Administrates system resources, and communicates with hardware. |

A layered archtitecture like the Android OS, ensures modularity, maintainability and security. Each layer is designed with a specific responsibility, and communicates primarly with adjacent layers. Applications interact with the Framework layer via the Android SDK (bridge between Application Layer and Framework Layer ), which provides access to core system services. The Framework, in turn, relies on the Native Libraries and Runtime layers for functionalities like graphics rendering, media playback, and app execution.

Hardware communication is abstracted through the Hardware Abstraction Layer (HAL), which interfaces with the Linux Kernel. This structured separation of concerns enhances the system's flexibility, allowing independent evolution and optimization of each layer.

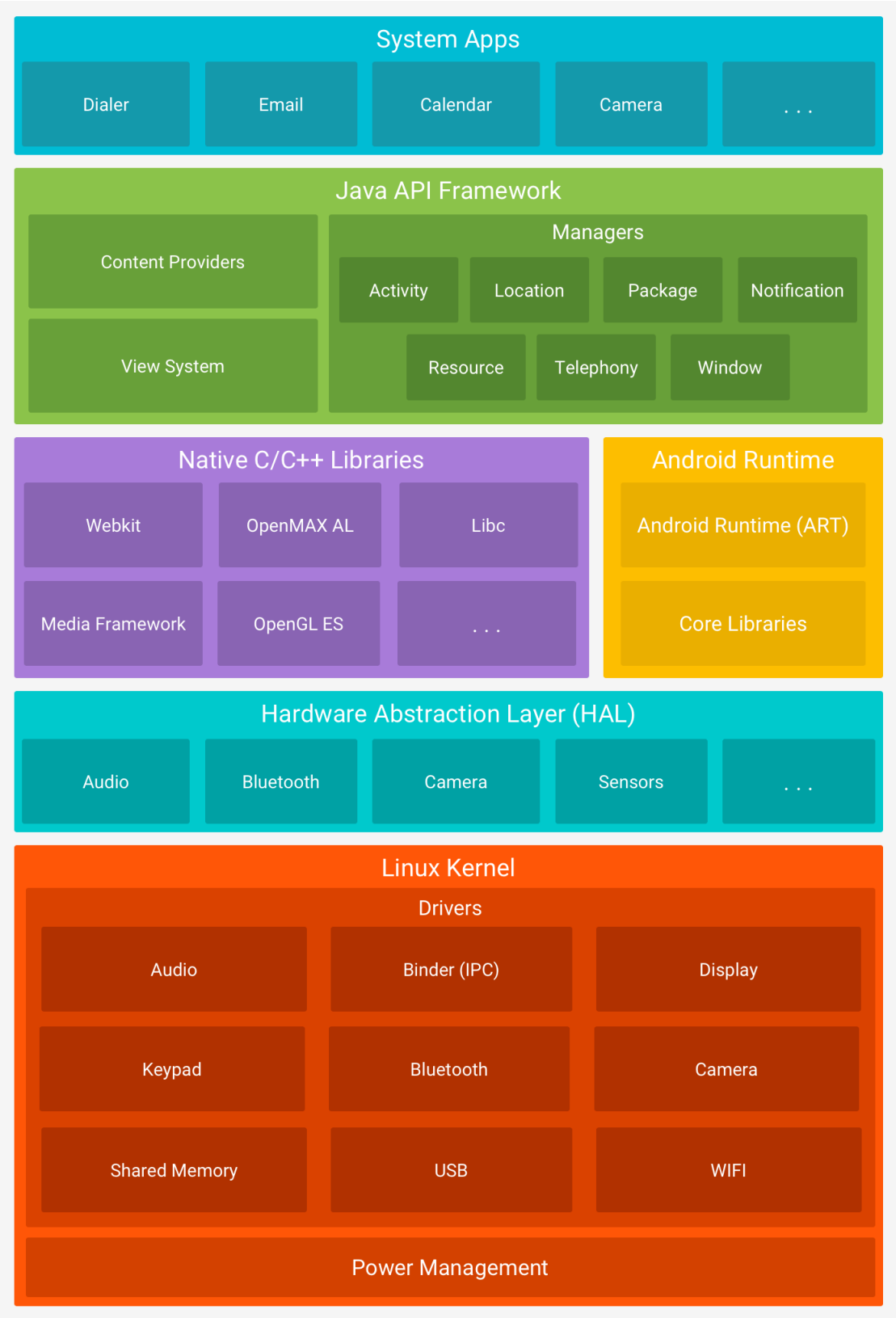



**Figure 1:** Android Platform architecture

### 2.1.1 Java Framework API (Framework Layer)

#### Description

As already mentioned in 2.1, the Android Java Framework API is a collection of libraries and tools provided by Android to simplify the development of mobile applications. Built on top of Java, it enables developers to create apps with pre-built components, utilities, and services that interact with the Android operating system. The framework abstracts the complexity of low-level operations and provides high-level APIs for features such as UI design, data storage, networking, and hardware access.

#### Project example

To provide an impression, Figure 2 shows a code excerpt in which the Java API was implicitly used by androidx. To keep this chapter manageable, it can be simplified said that androidx is a superset of the Java API Framework.

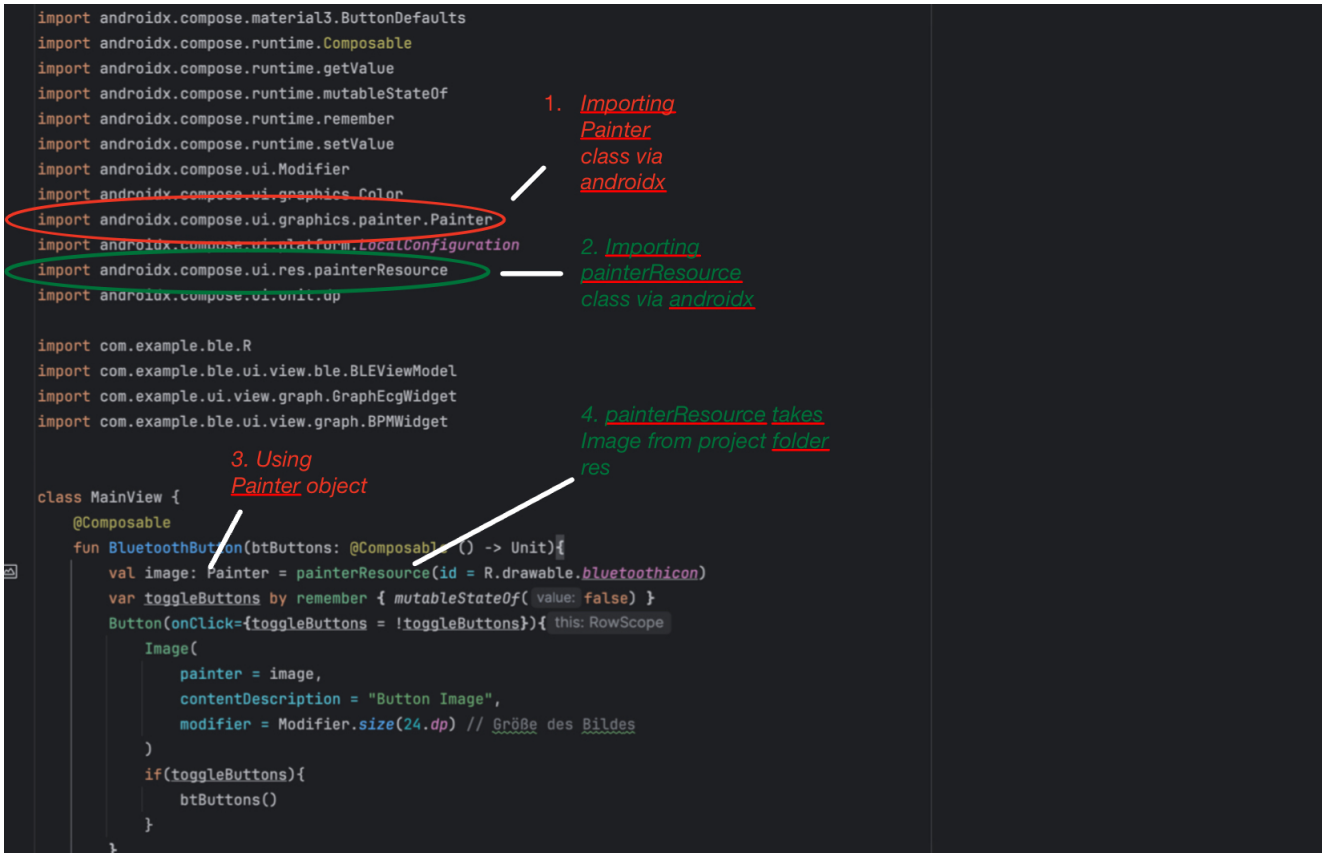



**Figure 2:** Android Java API example with androidx

Figure 2 provids a glimpse of the capabilities of the androidx library. Showing how to draw an image with the Painter class and utilizing the resource manager mentioned in table 1.

To beginn its elementary to import the Painter class aswell as the painterResource class. As shown in 1. and 2. in Figure 2. Then at 3. a Painter object is created, at 4. the Painter object is definied by the painterResource function. The painterResource function gets an id as an argument, to query the file from the res/drawable folder. The res folder contains all additional app resources (files, images, etc.). Finally we can draw the Image by passing the object from 3. to an Image function.

# 3 Functional Requirements

Requirement engineering is essential at the beginning of every project. Formulating and documenting precisely defined requirements is a cornerstone in the course of realization. In this way, mundane mistakes can be avoided and the vision of the final product is clear. Thus this chapter will explain all functional requirements of the finshed product.

## 3.1 ECG-Data

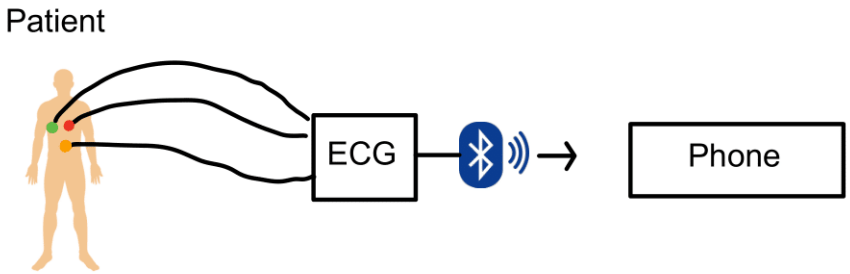



**Figure 3:** Informationtransfer

The application receives integer values from the microcontroller, which represent the heartbeat of the person connected to the hardware. These values must be mapped onto the screen to provide a clear and accurate visualization of the heart's activity. A key challenge in this process is ensuring that the visualization adapts to different screen sizes. Since the application may run on devices with varying display dimensions and resolutions, a static mapping approach is insufficient. Instead, a dynamic and scalable algorithm is required to transform the received data into a graphical representation that maintains consistent proportions and readability across all screen sizes. To achieve this, the application must implement a flexible coordinate transformation. The incoming data points need to be scaled appropriately based on the available screen width and height while maintaining the correct aspect ratio of the graph. Additionally, the algorithm should

support real-time updates to provide an accurate and responsive user experience. The transformation of raw data points must be performed quickly, otherwise, there is a risk of data loss. Therefore many parameters are need to be considerd when transforming the data.

**Parameters**

- Screen dimensions (height, width)
- Current min and max value in buffer
- Buffer capacity
- $\Delta v \quad , where\ \Delta v = \max - \min$ (range of values in current Buffer)
- coefficient $= \dfrac{\text{height} - 2 \times \text{shiftH}}{\Delta v}$ (maps raw values to screen height, with margin of shiftH pixels at the top and bottom)
- step $= \frac{width}{bufferCap.}$(Determines horizontal space for each data point)

all data in the buffer will be updated as shown in the following code snippet:

```
1    for (i in dataTemp.indices) {
2        data[i] = (maxV - dataTemp[i]) * coeff + shiftH
3    }
```

**Code 1:** Einfaches Beispiel eines Sourcecode-Generators, abgeleitet von **??** und eigenen Ergänzungen

The calculation process can be examined in greater detail in the following example, which demonstrates how each data point is transformed and mapped onto the screen.
To keep the example simple and concise, we will deal with an array of size 5 and a Coordinate system with a range of (6,6):

| 951 | 960 | 980 | 1050 | 930 |
|---|---|---|---|---|

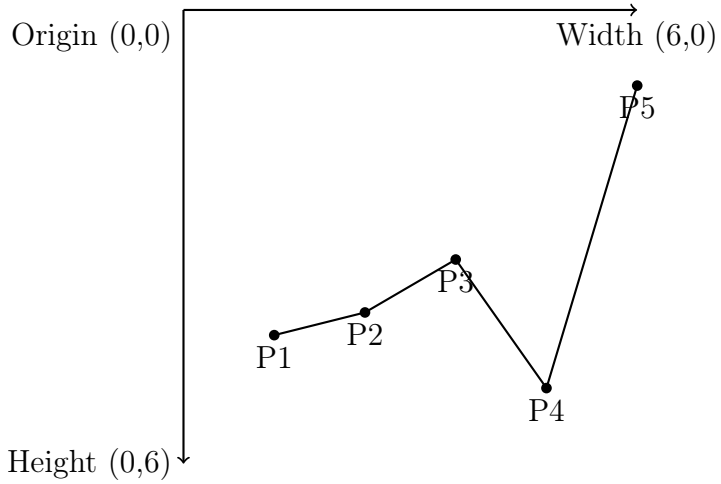
- Screen dimensions (6,6)
- minV = 930, maxV = 1050
- BufferCapacity = 5

- $\Delta v = 1050 - 930$
- coefficient $= \dfrac{6 - 2 \times 1}{120}$ (shiftH $= 1$ )
- step $= \frac{6}{5}$

The newly evaluated Array contains the following data:

| 4.4 | 4.0 | 3.3 | 1.0 | 5.0 |
|---|---|---|---|---|

Resulting in the follwing drawing:




Resulting in the follwing Complexity Analysis:

- Data Conversion: O(n) (for n data points)
- Scaling & Mapping: O(n) for n data points)
- Overall Complexity: O(n) linear (for n data points)

The resulting linear Complexity of O(n) makes the algorithm efficient for real-time processing.

## 3.2 Drawing ECG-Data in realtime

Processing real-time electrocardiographic (ECG) data presents multiple challenges that must be addressed to ensure accurate and efficient visualization. It is helpful to split this rather complex functionality into multiple concerns:

- The sample rate of the microcontroller (1)
- Consume and process incoming data simultaneously (2)
- Datahandling and Drawing in parallel (3)

These issues must be viewed and solved isolated, and subsequently combine each solved problem with the other. It is important to mention that some issues may create restrictions for other issues. Therefore, it is essential to implement one task at a time and connect them all afterward.

### (1) Handle The Sample rate of the microcontroller

The microcontroller sends 10 samples every 8 milliseconds (1000ms = 1s).

$$\frac{1000}{8} = \frac{125}{1} \mathrel{\widehat{=}} 125Hz$$

Hence we have to process a sample rate of: $f(t) = 125 \times t$, t in seconds. If handling this datastream is not possible, data will be lost.

### (2)Consume and process incoming data at the same time

To consume and process data at the same time, spliting up processing and consuming into two indepent tasks by assigning each task to its own thread, is necessary. Making it possible to operate in paralell.

Since both threads have to access the same ressource at the same time, we have to make use of mutual exclusion.

### (3)Datahandling and drawing in parallel

Thus the drawing entity has to countiously read data, and the datahandler entity receives a continous data stream. The suitable data structure must be able to constantly record new data and retain a certain amount of old data. So that a screen-width drawing can be displayed continuously. In addition, it is important that the data structure is not to large, which would lead to visible latencies.

This led to the Ringbuffer beeing the datastructure of choice. Ringbuffers are often used when having to process datastreams. The ringbuffer has a fixed size, thus old data gets overwritten. Therefore, there is no need to worry about the data structure consuming too much memory.

## 3.3 24-Hour recording

The functional requirements of the 24-Hour recording are shown in the list below. All of the listed requirements will be explainend in detail in chapter 4.

- Recording mode
- Filechooser conzept, to choose multiple files
- Start a recording, and assigning a name
- Abort a recording
- Display 24-Hour long recorded ECG-Data
- Show the current time intervalls
- Zoom into the displayed data, zooming out of the displayed data
- Plot BPM of the current displayed time intervall

## 3.4 Connection via Bluetooth Low Energy

The application is required to establish and maintain a reliable Bluetooth Low Energy (BLE) connection with the hardware. A stable connection is essential for continous data-transfer aswell as for the 24-Hour recording Mode. The BLE entity must be capable of the following items:

- Continous datastream
- Communicate with the application
- Share current the current mode (realtime, or recording)
- Scan for near devices
- split & reassamble to large datasets

# 4 Realization

The first part of this chapter provides an overview of the information flow between each entity, and the tasks of the given entitys. Aswell as an close look into the implementation of the functional requirements, by analysing code snippets, aswell as explaining the usage of the final product. While the second part will explain every necessity to assemble all parts together for a working product.

## 4.1 Selected Technologies

"The nature of the application and the required functionality of a software system should influence the choice of methods and tools used during development". Clarke [1995] This ensures that the development process is both efficient and tailored to meet the unique demands of the system. Moreover, aligning tools and methods with the application's requirements can help streamline workflows and reduce potential complexities during implementation. Consequently choosing the right programming language, aswell as IDE is essential to enable efficient development.

### 4.1.1 Programming Language and Framework

When searching for a suitable programming language to program Android apps, names like Java/Kotlin and Dart often appear. Both languages offer great frameworks, such as Flutter for Dart or Jetpack Compose for Kotlin. One aswell as the other are highly effective because they eliminate the limitations of XML-based UI development, enabling a more streamlined, declarative approach that combines UI and logic in a single language. Thus reduces boilerplate code, and enhances performance aswell as developer productivity. The main differnce between both frameworks lies in ressource manegement. For example Flutter is superior in cpu usage, but Jetpack Compose consumes less memory.Kusuma et al. [2023] "But since both have their respective advantages and disadvantages, it is impossible to say that one is superior to the other. The choice between Jetpack Compose and Flutter depends on the needs and preferences of the developer. If developers are more familiar with the Kotlin language and want to use new technologies, then Jetpack Compose might be a better choice." Kusuma et al. [2023] With prior practical experience in Java/Kotlin, Jetpack Compose was chosen as the framework for this work.

### 4.1.2 Development Enviroment

Integrated development Enviroments play a fundamental role in todays application development, providing developer with tools mandatory to debug, refactor and write code.

Making programming more efficient and delightful for programmers. When developing Android Apps with state of the art frameworks like Jetpack Compose, Android Studio stands out as the most suitable option as both Jetpack compose and Android Studio are created by Google. Android Studio is the official IDE for Android Development. With IntelliJ as base it inherits powerful code editor and developer tools, combined with Android specific features. Such as Live Edit (updates running app when code is changed) aswell as Interactive Preview which allows to test functionallity between so called Compose components (Compose will be explained in the later course of this work)[1], et cetera.

The version of Android Studio used for this thesis is **Ladybug | 2024.2.1**. Which is the latest version as this was written (13.12.2024).[2] Downloading the right version can be crucial for problem-free debuging. As some versions lead to bugs, causing the screen to freeze.[3]

## 4.2 Definiton of Entitys

As shown in table 2 each entity has its scope of dutys, making the final code more modular and consequently maintainable.

[1] https://developer.android.com/studio/intro
[2] https://developer.android.com/studio/releases
[3] https://stackoverflow.com/questions/78125635/processing-classes-for-emulated-method-breakpoints-this-popup-comes-and-loadin

**Table 2:** Entiys area of responsibility

| **Layer** | **Functionality** |
|---|---|
| BLEViewModel | - Communication with the microcontroller<br>- Receives datastream<br>- Sends instructions to microcontroller |
| GraphEcgWidget | Draws the ECG data in realtime,<br>and displays Beats per minute |
| Storewrapper | Converts data from microcontroller<br>into drawble data |
| BPM | Calculates the current Beats Per Minute |
| VitalMonitorWidet | Plots up to 24 Hour long recoreded ECG<br>data and the BPM of the plotted Intervall.<br>Provides zooming functionality. |
| StorageHandler | Allows usage of the OS filechooser,<br>to select files. |
| MainView | Collection of visble elements (Widgets),<br>to provide reusability |
| MainActivity | Visible to user, contains the complete<br>User Interface. Takes and processes user<br>inputs. |

### 4.3 Inter-Entity Communication Overview

The following diagramm shows the Information flow between all entitys. The unidirectional arrow means that, information is flowing oneside. An informationexchange between two entitys is represented by an bidirectional arrow.

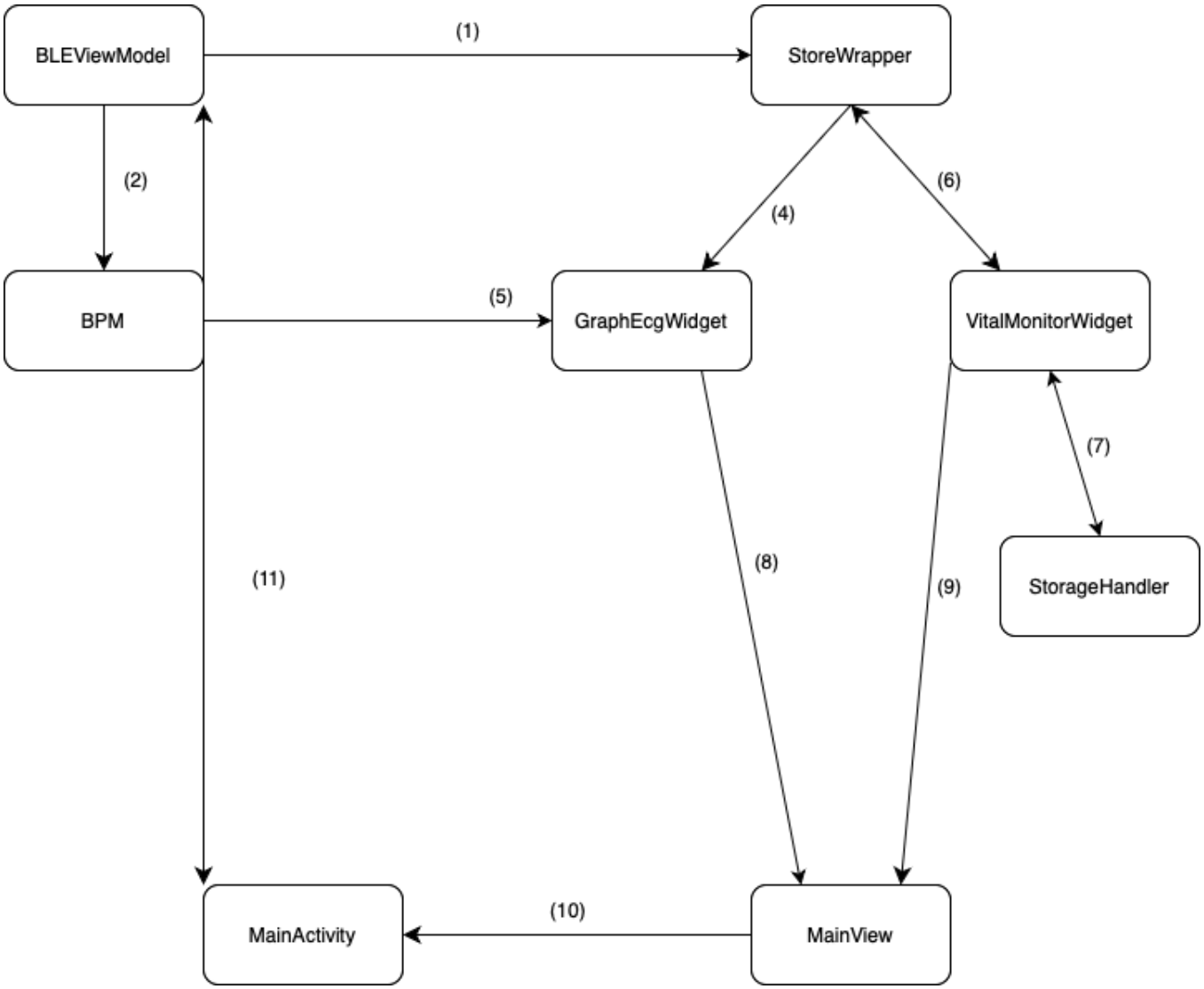



**Figure 4:** Informationflow Diagramm

#### Information Flow and Responsibilities of BLEViewModel

- **BLEViewModel as an Interface**
  - Acts as an interface between the controller and the app (4.2).

  - Any entity that evaluates recorded data depends on this instance.

- **Entities Dependent on BLEViewModel**
  - **StoreWrapper**
    - Makes the data drawable (1).
    - Forwards the data to the **GraphECGWidget** (4).
  - **BPM Instance**
    - Calculates the beat frequency (2).
    - The result is displayed by the **GraphECGWidget** (5).
- **Comparison of Information Flows (4) and (6)**
  - Both provide drawable data to widgets.
  - **Difference**:
    - **(4) One-way flow** → StoreWrapper receives data directly from the ECG.
    - **(6) Bidirectional flow** → Required for reading recorded data from a local file (7) using the **StorageHandler** before sending it to the StoreWrapper.
  - Once mapped by the StoreWrapper, the data is sent back to the **VitalMonitorWidget** (6).
- **MainView Responsibilities**
  - Integrates **GraphECGWidget** (8) and **VitalMonitorWidget** (9).
  - Manages the visibility of these components.
- **Extended BLEViewModel Responsibilities**
  - Beyond data exchange, BLEViewModel also:
    - Scans and connects to other nearby devices.
    - Integrates and manages 24-hour ECG recordings.
  - These functionalities are used by **MainView** and **MainActivity**.
- **MainView vs. MainActivity**
  - **MainView**
    - Constructs components and functionalities for **MainActivity**.
    - Provides UI elements, such as buttons for starting a 24-hour ECG recording (10).

- **MainActivity**
  - Acts as a bridge between the **microcontroller** and the **user**.
  - Handles device scanning, recording initiation, connection establishment, and ECG data display (11).

## 4.4 BLEViewModel

The following subsections explore the logical implementation and functionality of individual entities, using code snippets and application screenshots for illustration. Each subsection is divided into a section covering the implementation and another focusing on the application's usage, with both sections referring to corresponding screenshots. Framework specific code fragments will be explaing in the course of relevant section.

Before presenting the implementation of the BLE component, it is essential to establish a fundamental theoretical understanding of BLE. This section offers a comprehensive explanation of the Bluetooth Low Energy, shedding light on its core principles, while exploring the architecture of the BLE stack.

### 4.4.1 BLE Stack

The BLE protocol stack consists of three main layers: the Controller, the Host, and the Application. The Controller encompasses the Physical Layer and the Link Layer, while the Host manages higher-level functionalities such as the Logical Link Control and Adaptation Protocol (L2CAP), the Attribute Protocol (ATT), the Generic Attribute Profile (GATT), the Security Manager (SM), and the Generic Access Profile (GAP). Communication between the Host and the Controller follows the standardized Host Controller Interface (HCI). At the top of the stack, the Application layer enables program execution. Dian et al. [2018]

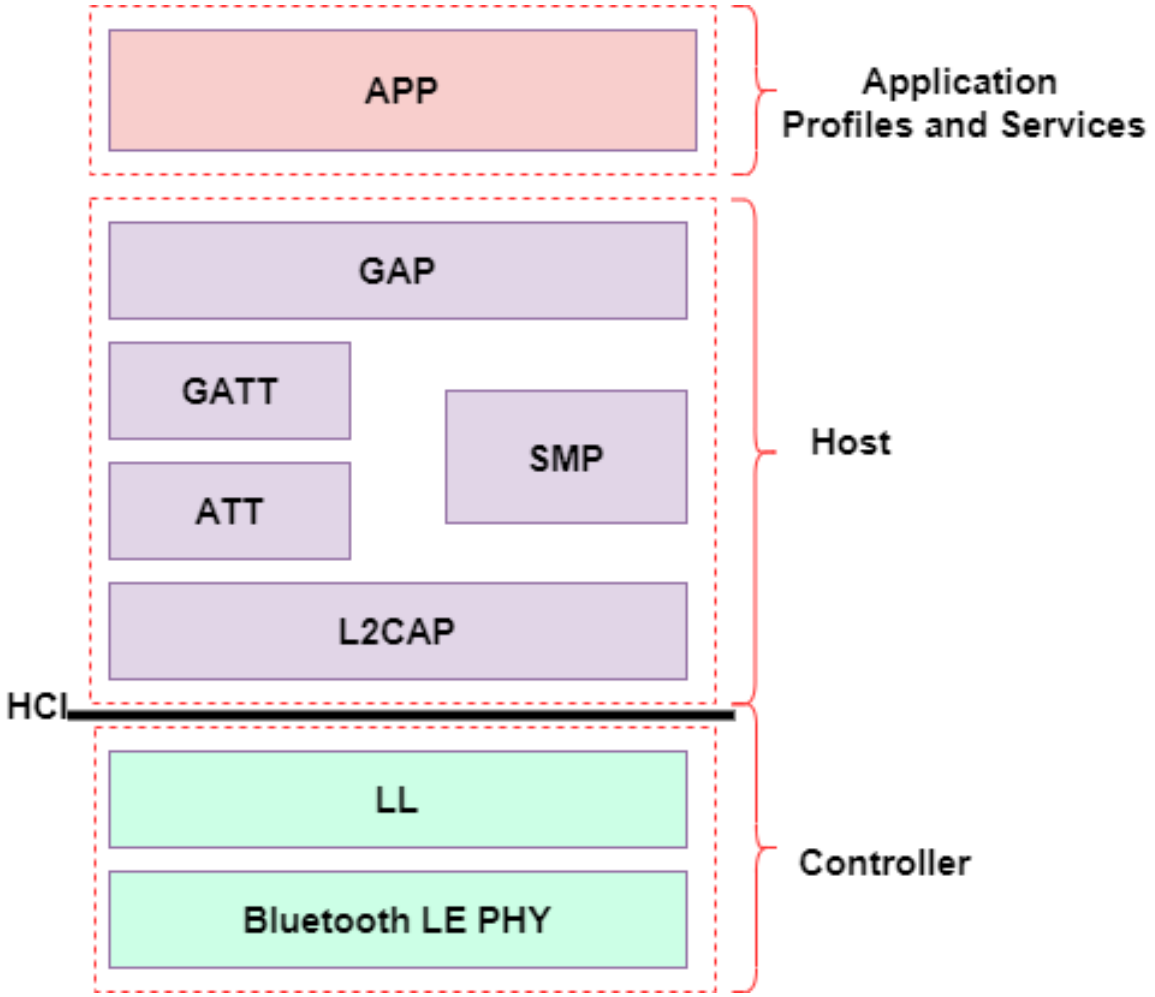



**Figure 5:** BLE protocol stack (https://de.mathworks.com/help/bluetooth/gs/bluetooth-protocol-stack.html)

### Application Layer

The application layer serves as the direct user interface, defining profiles that enable interoperability between various applications.

### Host Layer

Includes the host-sided HCI aswell as L2CAP, ATT, GATT, SMP and GAP.

### Generic Acces Profile (GAP)

Manages the connection setup aswell as security, interfaces directly with profiles and services of the Application Layer.

### Security Manager Protocol (SMP)

Provides multiple security algorithms for encrypting and decrypting data packets, making the connection secure.

### Generic Attribute Profile (GATT)

Defines services and characteristics provided by the GATT Server. The GATT Server in this case is the microcontroller, which holds the definitions of services and characteristics. These can be requested by the GATT Client (in this case the application). Services organize data into logical entities and contain specific data units known as characteristics. Each service can include one or more characteristics and is uniquely identified by a numeric UUID. This UUID can be either 16-bit for officially adopted BLE services or 128-bit for custom services. Whereas the Characteristic is the lowest level concept in GATT transactions, which defines data. Services aswell as Characterics are main interactions points to work with when working with the BLE periphery, thus it is important to understand these terminologies.

### Attribute Protocol (ATT)

The ATT enables the exchange of attributes (characteristics). Attributes are are composed of four components, the type (128-bit UUID), an handle (16-bit indentifier) to reference attributes, rights of the attribute(reading, writing), and the value of the attribute. The following figure gives a good overview of the structure of the attributes.

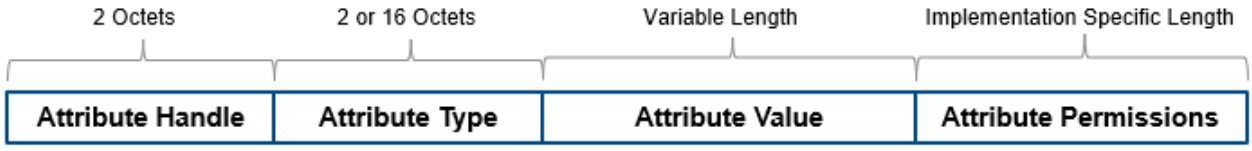



**Figure 6:** Attribute Structure (https://de.mathworks.com/help/bluetooth/gs/bluetooth-protocol-stack.html)

### Logical Link Control and Adaption Layer Protocol (L2CAP)

Works as an interface betwenn the HCI and the upper modules of the Host layer, by multiplexing data packets to the corresponding protocols. Also fragments and reassembles large packets.

#### Host-Side Host-Controller-Interface (HCI)

The HCI is the inteface between the host and the controller. The HCI establishes commands and events for packet data transmission and reception. When sending data, it processes raw information into packets, enabling communication between the host and the controller.

#### Controller-Side Host-Controller-Interface (HCI)

As mentioned in the last paragraph, the HCI is the interface between the host and the controller. However, analogous to the host side, the data received from the controller is extracted and forwarded to the host.

#### Controller Layer

Includes the Bluetooth Le PHY, LL, and controller-side HCI

#### Link Layer (LL)

The LL functions similarly to the medium access control (MAC) layer in the OSI model. In Bluetooth, it directly interacts with the Bluetooth LE PHY and controls the radio's link state, determining whether a device operates as a Central, Peripheral, Advertiser, or Scanner.

#### Bluetooth LE PHY

The physical layer (BLE PHY) is a 1 Mbps GFSK-based radio with adaptive frequency hopping, operating in the unlicensed 2.4 GHz ISM (industrial, scientific, and medical) band.

### 4.4.2 Implementation

#### Dependency versions and Library

Before delving into the implementation, it is essential to note that a library provided by Nordic Semiconductor was employed in the development process.[4] The utilized dependency version are shown in the figure below:

[4] https://github.com/NordicSemiconductor/Android-BLE-Library

```
implementation("no.nordicsemi.android.kotlin.ble:scanner:1.1.0")
implementation("no.nordicsemi.android.kotlin.ble:client:1.1.0")
implementation("no.nordicsemi.android.kotlin.ble:advertiser:1.1.0")
implementation("no.nordicsemi.android.kotlin.ble:server:1.1.0")
implementation ("no.nordicsemi.android:ble:2.4.0")
implementation ("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.6.0")
implementation("io.coil-kt:coil-compose:2.2.2")
implementation ("androidx.collection:collection:1.4.3")
```

**Figure 7:** Library Versions

### Scanning devices

The following code snippet demonstrates the implementation of a BLE scanning method used to detect nearby devices. The *startScan* function initializes the scanning process by setting the *scanState* to *SCANNING* and resetting the list of discovered BLE devices. The scanning operation is performed using *BleScanner*, with results processed by *BleScanResultAggregator*. The detected devices are filtered to include only those with the name *"THI-EKG"* and stored in a MutableState. The scan runs asynchronously within the viewModelScope, and after a delay of 2000 milliseconds, it is terminated by canceling *scanJob* and updating the *scanState* to *DONE*. The snippet also contains two annotations above the function definition, which dont add any functionality, but provide robustnes and readability to the code. Nevertheless, a brief explanation follows to avoid confusion and ensure clarification:

**@RequiresApi(Build.VERSION_CODES.UPSIDE_DOWN_CAKE)**

This annotation ensures that the *startScan* function is only executed on devices running Android API level **34 (Android 14, Upside Down Cake)** or higher. If the function is called on a lower API level, the compiler will issue a warning, and runtime execution may fail.

**@SuppressLint("MissingPermission")**

This suppresses lint warnings related to missing Bluetooth permissions (such as *BLUETOOTH_SCAN* or *ACCESS_FINE_LOCATION*). Normally, scanning for BLE devices requires specific permissions, and Android Studio would generate a warning if they are not explicitly checked. However, this annotation tells the lint tool to ignore such warnings, assuming the necessary permissions are already handled elsewhere in the code. Thus the permissions are check in the Main Acitivity entitiy, this warning can be ignored, and will appear more often.

```
1    private var scanJob: Job? = null
2
```

```
    @RequiresApi(Build.VERSION_CODES.UPSIDE_DOWN_CAKE)
    @SuppressLint("MissingPermission")
    private fun startScan(context: Context) {
        scanState.value = ScanState.SCANNING
        foundBLEDevices.value = mutableSetOf()

        val aggregator = BleScanResultAggregator()
        scanJob = BleScanner(context).scan()
        .map { aggregator.aggregateDevices(it) }
        .onEach {
            foundBLEDevices.value = it.stream()
            .filter { it.name == "THI-EKG" }
            .collect(Collectors.toSet())
        }
        .launchIn(viewModelScope)

        viewModelScope.launch {
            delay(2000)
            scanJob?.cancel()
            scanState.value = ScanState.DONE
        }
    }
```

**Code 2:** Code for scanning functionality

### Refreshing

Before initiating the BLE scan, the function checks for two critical permissions:

- BLUETOOTH_SCAN: This permission is required to scan for nearby BLE devices.
- BLUETOOHTH_CONNECT: This permission is necessary to establish a connection with discovered BLE devices.

The function uses ActivityCompat.checkSelfPermission to verify whether these permissions have been granted by the user. This step is crucial for ensuring that the application complies with Android's permission model, which mandates explicit user consent for accessing sensitive device features. Conditional Execution: If both permissions (BLUETOOTH_SCAN and BLUETOOTH_CONNECT) are granted, the function proceeds to call startScan(context). If the permissions are not granted, the function includes a TODO comment, indicating that additional logic is required to handle this scenario. Typically, this would involve prompting the user to grant the necessary permissions through a permission request dialog.

```
    @RequiresApi(Build.VERSION_CODES.UPSIDE_DOWN_CAKE)
    fun refreshBLE(context: Context) {
```

```
        if (ActivityCompat.checkSelfPermission(
        context,
        Manifest.permission.BLUETOOTH_SCAN
        ) == PackageManager.PERMISSION_GRANTED &&
        ActivityCompat.checkSelfPermission(
        context,
        Manifest.permission.BLUETOOTH_CONNECT
        ) == PackageManager.PERMISSION_GRANTED
        ) {
            startScan(context)
        }
    }
```

**Code 3:** Code for refresh functionality

### Connect

The provided BLE implementation is responsible for establishing a connection with a BLE-enabled device, receiving real-time data from specific BLE characteristics, and processing the incoming data to extract and analyze electrocardiographic (ECG) information. The functionality can be divided into the following key components:

**1. BLE Connection Setup (Line 143-149)**

The function connect initializes the connection with a given BLE device. Using Client-BLEGatt.connect, a GATT connection is established, and the discovered services and characteristics are retrieved. The relevant characteristics include:

*signalCharacteristic:* Used for receiving real-time electrocardiographic data.
*appCommunicationCharacteristic:* Used for bidirectional communication with the BLE device. The obtained appCommunicationCharacteristic is stored in the BLEViewModel for further use.

**2. Receiving and Processing Data from the Signal Characteristic**

The signalCharacteristic, continuously provides data in the form of a ByteArray. Each received data packet undergoes the following procedure:

The received byte array is first converted into a list of integers. To enable concurrent processing, the data is then transmitted through a channel (channel.send(values)). A coroutine, executed by receiveData(viewModelScope), retrieves the streamed data from the channel and passes it to StoreWrapper.processDataList(datastream). (Since courotines and channels play a central role in this and other implementations, this chapter is followed by a brief introduction to these Kotlin-specific topics.) The processed data is also added to the BPM.currList buffer for frequency analysis. Once the buffer exceeds 7500 samples, the QRS peak detection algorithm (Pan-Tompkins) is applied to identify heartbeats. The

detected peak count is logged for debugging, and to maintain a sliding window approach, the oldest 750 samples are discarded.

**3. Handling Application-Level BLE Communication**

Notifications are also enabled for the appCommunicationCharacteristic, which allows bidirectional communication with the BLE device. Each received notification undergoes:

Validation using ResponseChecker.check. Packet reconstruction via PacketReassembler.reassemble. Logging of the received data for debugging purposes.

**4. Additional Commands**

The implementation also includes:

Sending an MTU request (requestMtu(512)) to maximize packet size for efficient data transmission. Checking if the application is in live mode or in 24-Hour mode by calling fetchCurrentMode(). Also feteching all recorded ECG's from the microcontroller ListEKGS().

***Channels and Coroutines***

The *channe*l serves as a concurrent data pipeline between the BLE data stream and the processing logic. It ensures that the incoming data from the BLE device is processed in a structured, thread-safe manner without blocking the main execution thread. By leveraging coroutines, the channel allows efficient real-time data handling while preventing performance bottlenecks.

A *coroutine* in Kotlin is a lightweight, non-blocking way to handle asynchronous programming. It allows you to write code that can be paused and resumed without blocking the main thread, making it ideal for handling tasks like network requests, BLE communication, or large computations without freezing the UI.

```
1    @SuppressLint("MissingPermission")
2    fun connect(context: Context, device: ServerDevice) {
3        this.viewModelScope.launch {
4            val connection = ClientBleGatt.connect(context, device,
                 this@BLEViewModel.viewModelScope )
5            this@BLEViewModel.connection.value = connection
6            val services = connection.discoverServices()
7            val bleService = services.findService(UUID.fromString("e28e00e9
                 -b79a-479a-a6f1-21e8f73236e1"))
8            val signalCharacteristic = bleService!!.findCharacteristic(UUID
                 .fromString("23b85b26-e7cb-421c-9731-1b4ca2d2a849"))
9            val appCommuncationCharacteristic = bleService!!.
                 findCharacteristic(UUID.fromString("563a2846-1dc7-11ef
                 -9262-0242ac120002"))
10           this@BLEViewModel.appCommuncationCharacteristic =
                 appCommuncationCharacteristic!!
```

```
11
12              signalCharacteristic!!.getNotifications().onEach {
13                  val values = byteArrayToIntArray(it.value).toList()
14                  startTime = System.currentTimeMillis()
15                  channel.send(values)
16                  receiveData(viewModelScope){dataList ->
17                      StoreWrapper.processDataList(dataList)
18                      BPM.currList.value += dataList
19                      if(BPM.currList.value.size >= 7500){
20                          var qrsPeaks = BPM.panTompkins(BPM.currList.value).
                                size
21
22                          BPM.currList.value.forEach {
23
24                          }
25
26                          BPM.currList.value = BPM.currList.value.subList
                                (750, 7500-1)
27                          BPM.countPeak = qrsPeaks
28                          qrsPeaks = 0
29                      }
30                  }
31                  paketCounter++
32              }.launchIn(viewModelScope)
33              appCommuncationCharacteristic!!.getNotifications().onEach {
34                  ResponseChecker.check(it.value)
35                  PacketReassembler.reassemble(it.value)
36
37              }.launchIn(viewModelScope)
38              connection.requestMtu(512)
39              fetchCurrentMode()
40              listEKGS()
41          }
42      }
```

**Code 4:** Einfaches Beispiel eines Sourcecode-Generators, abgeleitet von **??** und eigenen Ergänzungen

## 4.5 Storewrapper

The StoreWrapper class serves as a central component for processing and managing electrocardiographic (ECG) data in real-time. It is responsible for converting raw data received from the BLE signal characteristic into a format suitable for visualization. This

transformation ensures that the incoming stream of ECG values can be efficiently rendered within the application's graphical interface. A key feature of the StoreWrapper class is its mapping functionality, which translates raw numerical values into drawable data points. This conversion is essential for accurately representing the ECG signal on-screen. Additionally, the class incorporates a ringbuffer, which provides an mechanism for storing and handling continuous data streams. The ringbuffer ensures that only the most recent samples are retained, enabling a sliding window approach for real-time signal visualization and processing.

### 4.5.1 Implementation

#### Preparing ECG-Data

The provided code snippet is responsible for transforming raw electrocardiographic (EKG) data into a format suitable for visualization across different screen sizes. This transformation ensures that the signal is correctly scaled and displayed within the graphical interface, regardless of resolution or aspect ratio. The function achieves this with a time complexity of O(n), making it efficient for real-time applications.

The function begins by initializing the shiftH parameter and preparing a mutable list (data) for storing processed values. If an external data source is provided, it is converted from a generic format into a list of Double values. Otherwise, an empty list is created.

**Screen Size and Scaling Factors**

The width and height of the target display area are extracted from the size parameter. Additionally, the function determines the minimum (minV) and maximum (maxV) values within the buffer to establish the dynamic range of the data. If all values are equal, an artificial spread is introduced to prevent division by zero.

**Normalization and Scaling**

The difference between maxV and minV (dv) is calculated to determine the range of data values. A step size (step) is then computed based on the available screen width and the buffer capacity, ensuring that data points are evenly distributed across the display. The coeff value is derived to scale the signal to fit within the available height while accounting for a vertical shift (shiftH).

**Data Mapping for Graph Rendering**

A temporary list (dataTemp) holds the normalized data, which is then mapped to a new list (data). The mapping follows the formula:

$data[i] = (maxV\text{-}dataTemp[i]) \times coeff + shiftH$

This equation ensures that the data is scaled appropriately for graphical representation. The function iterates over the dataset in a single pass (O(n) complexity), making it highly efficient for real-time applications. This guarantees smooth and responsive rendering, even with large data sets.

```
fun prepareData(size: Size, shiftH: Double, dataFromFile:
    MutableList<Any>? = null): List<Double> {
    this.SHIFTH = shiftH
    var data: MutableList<Double>? = null
    if( dataFromFile != null ){
        data = dataFromFile.mapNotNull {
            (it as? String)?.toDouble()
        }.toMutableList()
    }else{
        data = ArrayList()
    }
    val width = size.width.toDouble()
    val height = size.height.toDouble()
    var minV = min //lowest value in buffer
    var maxV = max  //highest value in buffer
    if (minV == maxV) {
        minV = minV / 2
        maxV = maxV + minV / 2
    }
    val dv = maxV - minV
    step = width / buffer_.capacity()
    val coeff = (height - 2 * shiftH) / dv

    val dataTemp =
    if (_mode == GraphMode.overlay) utils_.dataSeriesOverlay(
        buffer_) else utils_.dataSeriesNormal(
    this
    )
    data = ArrayList(Collections.nCopies(dataTemp.size, 0.0))
    //return data
    if (data.isEmpty() || dataTemp.isEmpty()) {
        return data
    }
    for (i in dataTemp.indices) {
        data[i] = (maxV - dataTemp[i]!!) * coeff + shiftH
    }
    return data
```

```
36          }
```

**Code 5:** Code for mapping data on the screen

### Preparing the Drawpath

To achieve a smooth and natural representation of the ECG waveform, it is essential to create two separate paths: one for the data points before the current write position in the buffer and one for the data points after. This ensures a seamless transition between old and new data, preventing visual gaps and making the waveform appear continuous, just like a real ECG signal.

The function **preparePathBefore(data: List<Double>)** constructs the path for the portion of the data preceding the current write index of the buffer. It determines the correct starting index and iterates through the data points up to the write position, generating a Path object that maps the signal to drawable coordinates.

Conversely, the function **preparePathAfter(data: List<Double>)** handles the data following the write index. It ensures that the path continues from the last drawn point, maintaining a smooth flow of the signal. By splitting the waveform into these two sections, the transition between old and new data is visually imperceptible, preserving the real-time appearance of the ECG signal.

This approach is crucial for maintaining a continuous and realistic ECG display, as it eliminates abrupt jumps or gaps in the graph, ensuring that the visualization remains fluid and accurately represents the ongoing signal dynamics.

```
1           fun prepareData(size: Size, shiftH: Double, dataFromFile:
                MutableList<Any>? = null): List<Double> {
2               this.SHIFTH = shiftH
3               var data: MutableList<Double>? = null
4               if( dataFromFile != null ){
5                   data = dataFromFile.mapNotNull {
6                       (it as? String)?.toDouble()
7                   }.toMutableList()
8               }else{
9                   data = ArrayList()
10              }
11              val width = size.width.toDouble()
12              val height = size.height.toDouble()
13              var minV = min //lowest value in buffer
```

```
14                var maxV = max  //highest value in buffer
15                if (minV == maxV) {
16                    minV = minV / 2
17                    maxV = maxV + minV / 2
18                }
19                val dv = maxV - minV
20                step = width / buffer_.capacity()
21                val coeff = (height - 2 * shiftH) / dv
22                //passing the wrong buffer to dataSeriesOverlay (not updated)
23                val dataTemp =
24                if (_mode == GraphMode.overlay) utils_.dataSeriesOverlay(
                      buffer_) else utils_.dataSeriesNormal(
25                this
26                )
27                data = ArrayList(Collections.nCopies(dataTemp.size, 0.0))
28                //return data
29                if (data.isEmpty() || dataTemp.isEmpty()) {
30                    return data
31                }
32                for (i in dataTemp.indices) {
33                    data[i] = (maxV - dataTemp[i]!!) * coeff + shiftH
34                }
35                return data
36            }
```

**Code 6:** Einfaches Beispiel eines Sourcecode-Generators, abgeleitet von **??** und eigenen Ergänzungen

```
1         fun preparePathBefore(data: List<Double>): Path {
2             var idx_ = buffer_.writeIndex() - 1
3             if (idx_ >= data.size) {
4                 idx_ = data.size - 1
5
6             }
7             val idx = if (idx_ < 0) 0 else idx_
8             val path = Path()
9             path.moveTo(0f, data[0].toFloat())
10            for (i in 1 until idx - 1) {
11                path.lineTo((i * step).toFloat(), data[i].toFloat())
12            }
13            return path
14        }
15
16        fun preparePathAfter(data: List<Double>): Path {
17            var idx_ = buffer_.writeIndex()
18            if (idx_ >= data.size) {
```

```
                idx_ = data.size - 1
            }
            val idx = if (idx_ < buffer_.capacity() - 1) idx_ else buffer_.
               capacity() - 1
            val path = Path()
            path.moveTo((idx * step).toFloat(), data[idx].toFloat())
            for (i in idx + 1 until data.size) {
                path.lineTo((i * step).toFloat(), data[i].toFloat())
            }
            return path
        }
```

**Code 7:** Einfaches Beispiel eines Sourcecode-Generators, abgeleitet von **??** und eigenen Ergänzungen

## 4.6 GraphEcgWidget

This code implements a real-time ECG visualization. The code renders ECG data as a graph, calculates the heart rate (BPM), and classifies it into different heart rate categories. Since the code of this class is very extensive, some code snippets are presented in abbreviated form.

### Displaying Heart Rate

The function *ECGValues(frequency: Int)* displays the current BPM and classifies it into one of three categories:

- **Bradycardia** (heart rate $<$ 60 BPM)
- **Normal Frequency** (heart rate 60-99 BPM)
- **Tachycardia** (heart rate $\geq$ 100 BPM)

```
@Composable
fun ECGValues(frequency: Int){
    ...
    when {
        frequency > 0 && frequency < 60 -> { /*     Bradycardia */ }
        frequency in 60..99 -> { /* Normal frequency */ }
        frequency >= 100 -> { /* Tachycardia */ }
        }
    ...
}
```

**Code 8:** Einfaches Beispiel eines Sourcecode-Generators, abgeleitet von **??** und eigenen Ergänzungen

### Continous ECG-Graph

The overlay graph ensures that the ECG waveform remains continuous by storing and displaying previous data points alongside the latest ones, rather than just showing the most recent signal. This prevents data loss and allows a smooth, uninterrupted visualization of the heartbeat over time. The drawOverlayGraph() function is responsible for rendering a continuous ECG graph by maintaining both past and current data. Instead of only displaying the most recent values, it preserves previously recorded data, ensuring that the graph remains complete and does not lose old information.

First, the function retrieves two paths:

pathBefore, which contains the past ECG data, ensuring historical values remain visible. pathAfter, which holds the newest ECG data points, representing the latest heartbeat progression. To ensure a clear visualization, the function checks whether there is a stored point from the latest ECG reading. If a point exists, a transparent rectangle is drawn at its position, keeping the background clean for the rendering process.

Then, both paths are drawn sequentially:

pathBefore is drawn in red with a thin stroke, representing the past ECG signal. pathAfter is drawn with a slightly thicker red stroke, making the latest ECG values more prominent. Finally, the function highlights the most recent ECG data point by drawing a red dot at the latest position, helping users identify the real-time measurement. By continuously displaying both past and new data, the overlay graph creates a seamless ECG visualization, allowing for smooth tracking of the heart's electrical activity over time.

```
1        fun drawOverlayGraph() {
2            val pathBefore = if (storeWrapper.pathBefore == null) Path()
                 else storeWrapper.pathBefore!!
3            val pathAfter = if (storeWrapper.pathAfter == null) Path() else
                 storeWrapper.pathAfter!!
4
5            if (storeWrapper.point != null) {
6                drawRect(
7                color = Color.Transparent,
8                topLeft = Offset(storeWrapper.point!!.x, 0f),
9                size = Size(width, height),
10               style = Fill
11               )
```

```
12              }
13
14              drawPath(
15              path = pathBefore,
16              color = Color.Red,
17              style = Stroke(width = 1f),
18              )
19
20              drawPath(
21              path = pathAfter,
22              color = Color.Red,
23              style = Stroke(width = 2f),
24              )
25
26              if (storeWrapper.point == null) return
27
28              drawCircle(
29              center = Offset(storeWrapper.point!!.x, storeWrapper.point!!.y)
                    ,
30              radius = 6f,
31              color = Color.Red
32              )
33          }
```

**Code 9:** Einfaches Beispiel eines Sourcecode-Generators, abgeleitet von **??** und eigenen Ergänzungen

### Real-time graph update

The *doneClick* function is responsible for starting or stopping the ECG data visualization when the user interacts with the graph. It first toggles the *isCoroutineRunning* flag to determine whether the coroutine should be started or stopped. It then calls *toggleCoroutine*, passing the new state along with the coroutine scope and the number of cycles. Within the coroutine, the function updates startTime with the current system time to measure performance and increments currentCounter to recompose the UI. If the counter is greater than zero, it prepares the drawing by calling *storeWrapper.prepareDrawing*, passing the screen width and height along with a fixed scaling factor. This ensures that the visualization area is correctly set up for rendering. Finally, the function updates heartFrequency by retrieving the latest BPM value from BPM.countPeak, ensuring that the displayed heart rate is continuously updated with the latest processed data. This function plays a crucial role in managing real-time ECG visualization, controlling when data processing runs and ensuring the graph updates dynamically as new data arrives.

```
1         fun  doneClick(cycles: Int) {
2            val width = screenWidth
3            val height = screenHeight
4            isCoroutineRunning = !isCoroutineRunning
5            toggleCoroutine(isCoroutineRunning, coroutineScope, cycles) {
                counter ->
6               currentCounter = counter + 1
7               if(counter > 0){
8                   storeWrapper.prepareDrawing(android.util.Size(width.
                       toInt(), height.toInt()), 32.toDouble())
9               }
10              heartFrequency = BPM.countPeak
11           }
12        }
```

**Code 10:** Einfaches Beispiel eines Sourcecode-Generators, abgeleitet von **??** und eigenen Ergänzungen

The toggleCoroutine function controls the execution of a coroutine that processes ECG data. It takes a boolean isRunning to determine whether the coroutine should start or stop, a coroutineScope to manage coroutine execution, an integer cycles, and a callback function that receives an integer value. When isRunning is true, a new coroutine is launched, and a channel is created to continuously send incremented counter values at an interval of 8 milliseconds. These values are then passed to the callback function, ensuring that the ECG data is updated in real time. If isRunning is false, all active child coroutines within the given scope are canceled, stopping data processing. This function ensures controlled and efficient execution of real-time ECG data handling, starting and stopping dynamically based on user interaction.

```
1         fun toggleCoroutine( isRunning: Boolean, coroutineScope:
             CoroutineScope, cycles: Int, callback: (Int) -> Unit) {
2            coroutineScope.launch {
3                if (isRunning) {
4                    val channel = Channel<Int>()
5                    launch {
6                        while (true) {
7                            channel.send(counter++)
8                            delay(8)
9                        }
10                   }
11                   for(value in channel){
12                       callback(value)
13                   }
14               } else {
```

```
15                      coroutineScope.coroutineContext.cancelChildren()
16                  }
17              }
18          }
```

**Code 11:** Einfaches Beispiel eines Sourcecode-Generators, abgeleitet von **??** und eigenen Ergänzungen

## 4.7 VitalMonitorWidget

The VitalMonitorWidget serves as a comprehensive visualization tool for mapping recorded 24-hour ECG data alongside the corresponding BPM values. It incorporates extensive functionality from the GraphECGWidget, leveraging its graphing capabilities to ensure smooth and accurate data representation. Additionally, it features interactive zooming functionality, allowing users to focus on smaller time windows where BPM values align precisely with the corresponding ECG waveforms. The data displayed within the widget is sourced from the file chooser, which is discussed in detail later in the thesis.

### Draw recorded BPM

The drawFlowingGraph function is responsible for rendering the ECG graph dynamically based on real-time data. It first checks if there is available data and ensures the counter is greater than zero before proceeding. It then determines the minimum and maximum values within the data set to establish the graph's vertical range. The function calculates the indices defining the portion of the data to be displayed, ensuring the graph remains within the zoomed-in section while allowing for user interactions such as tapping and zooming. A sublist of the ECG data is extracted, and if all values are equal, adjustments are made to ensure proper scaling.

To map the ECG data onto the graph, the function calculates a step size for horizontal placement and applies a transformation coefficient to scale the vertical axis. It then constructs a Path object, iterating through the extracted data points and converting each value into a corresponding position on the graph. The path begins at the first data point and continues connecting subsequent points to form a continuous line, representing the heart rate variation over time. Finally, the path is drawn onto the canvas in red with a defined stroke width, ensuring that the ECG waveform is clearly visible and responsive to real-time changes.

```
1  fun drawFlowingGraph() {
2      if(currentCounter > 0 && !dataFromFileBPM.isEmpty() && num == 0){
```

```
3         var minV = dataFromFileBPM.minOf { it.toString().toFloat() }
4         var maxV = dataFromFileBPM.maxOf{ it.toString().toFloat() }
5         fromIndexBPM =  if (tappedPointerIndexBPM - zoomValue <= 0)   0
              else tappedPointerIndexBPM-zoomValue
6         toIndexBPM = if (tappedPointerIndexBPM + zoomValue >=
              dataFromFileBPM.size) dataFromFileBPM.size else if (
              tappedPointerIndexBPM == 0)  dataFromFileBPM.size else
              tappedPointerIndexBPM + zoomValue
7         fromTime = fromIndexBPM
8         toTime = toIndexBPM
9         currentBPMList = dataFromFileBPM.subList(fromIndexBPM, toIndexBPM)
10
11
12
13        if (minV == maxV) {
14            minV = minV / 2
15            maxV = maxV + minV / 2
16        }
17
18        val dv = maxV-minV
19        step = width / currentBPMList.size
20        val coeff = (convertDpToFloat(density,150.dp) - 2 * shiftH) / dv
21        var path = Path().apply{
22            currentBPMList.forEachIndexed{i, point ->
23                var preparedPoint= 0.0f
24                if (maxV != null) {
25                    preparedPoint = ((maxV - point.toString().toFloat()) *
                          coeff + shiftH).toFloat()
26                }
27
28                if(i == 0 ){
29                    moveTo((i*step).toFloat(), preparedPoint.toFloat())
30                }else{
31                    lineTo((i*step).toFloat(),preparedPoint.toFloat())
32                }
33                currentCounter++
34            }
35        }
36        drawPath(
37        path = path,
38        color = Color.Red,
39        style = Stroke(width = 2f), // Stroke width
40        )
41        ...
42    }
```

**Code 12:** Einfaches Beispiel eines Sourcecode-Generators, abgeleitet von **??** und eigenen Ergänzungen

### Draw recorded EKG-Data

The second part of the drawFlowingGraph function is responsible for rendering ECG data with zooming functionality,and detailed visualization of the recorded signals. It first ensures that the ECG data is available and that the current counter and tapped pointer values are valid. The function determines the range of data points to be displayed by defining fromIndex and toIndex, ensuring that the graph remains within the available dataset while focusing on a smaller segment around the tapped pointer.

To scale the ECG values appropriately for rendering on the screen, the function calculates the minimum and maximum values of the selected data segment. If these values are equal, a small offset is applied to prevent division errors. The difference between these values (dv) is then used to determine a scaling coefficient (coeff), ensuring that the ECG waveform fits proportionally within the available drawing space.

The function then constructs a Path object (path2) by iterating through the selected ECG data points and mapping them to screen coordinates. The step variable determines the spacing between each point, ensuring that the waveform is evenly distributed along the horizontal axis. Each data point is transformed using the computed coefficient, adjusting its vertical position relative to the maximum value. The function begins the path with a moveTo operation for the first point and continues with lineTo operations to connect subsequent points, forming a continuous ECG waveform.

By dynamically selecting the data range based on user interaction and scaling the values appropriately, this function enables smooth zooming functionality. Users can focus on specific sections of the ECG signal while maintaining accurate and proportional visualization of the waveform.

```
1        ...
2        //POINTS OF EKG
3        if( num == 1 && !dataFromFilePoints.isEmpty() && currentCounter > 0
             && tappedPointer> 0){
4            var fromIndex =  if (tappedPointer-500 <= 0)   0  else
                 tappedPointer-500
5            var toIndex = if (tappedPointer+ 500 >= dataFromFilePoints.size
                 ) dataFromFilePoints.size else tappedPointer+ 500
6            var minV = dataFromFilePoints.minOf { it.toString().toFloat() }
7            var maxV = dataFromFilePoints.maxOf{ it.toString().toFloat() }
```

```
8               if (minV == maxV) {
9                   minV = minV / 2
10                  maxV = maxV + minV / 2
11              }
12              val dv = maxV-minV
13              step = width / dataFromFilePoints.subList(fromIndex, toIndex).
                    size
14              val coeff = (convertDpToFloat(density,150.dp) - 2 * shiftH) /
                    dv
15              var path2 = Path().apply{
16                  dataFromFilePoints.subList(fromIndex, toIndex).
                        forEachIndexed{i, point ->
17
18                      var preparedPoint = 0.0f
19
20                      if (maxV != null) {
21                          preparedPoint = ((maxV - point.toString().toFloat()
                                ) * coeff + shiftH).toFloat()
22                      }
23                      if(i == 0 ){
24                          moveTo((i*step).toFloat(), preparedPoint.toFloat())
25                      }else{
26                          lineTo((i*step).toFloat(),preparedPoint.toFloat())
27                      }
28                      currentCounter++
29                  }
30
31
32              }
```

**Code 13:** Einfaches Beispiel eines Sourcecode-Generators, abgeleitet von **??** und eigenen Ergänzungen

### Zooming functionality

This code implements a zooming functionality for the ECG visualization based on user tap interactions. The zooming behavior is controlled through single and double taps, which adjust the zoom level and update the reference point for the displayed data.

When a double-tap occurs, the zoom value increases by 150, effectively zooming into a more detailed view of the ECG data. The tappedPointerIndexBPM is then set to the center of the BPM data list, ensuring that the zoom focuses on the middle of the recorded data.

When a single tap is detected, the function adjusts the zoom level while updating the tappedPointer position based on the x-coordinate of the tap event. The relativeTap variable normalizes the tap position within the screen width, allowing for accurate mapping between the screen coordinates and the data indices. The currentTappedPointerIndexBPM is then calculated relative to the currently displayed BPM list, and tappedPointerIndexBPM is updated to reflect the corresponding index in the original dataset.

The zoom value is dynamically adjusted, decreasing by 30 with each tap until it reaches a minimum of 15, ensuring a smooth and controlled zooming experience. Additionally, the currentCounter is incremented, triggering updates in the visualization. This approach allows users to interactively zoom in and out of the ECG graph by tapping, providing a more detailed or broader view of the data as needed.

```
        .pointerInput(Unit) {

            // This makes the rectangle change color when touched (for touch input)
            detectTapGestures (onDoubleTap = {
                zoomValue += 150
                tappedPointerIndexBPM = dataFromFileBPM.size/2
                println("zoomVal: ${zoomValue}")



            }, onTap = {point ->
                if (zoomValue >= 150){
                    zoomValue = 150
                }
                tappedPointer = point.x.toInt()
                var relativeTap = tappedPointer.toFloat() / 720
                println("This is where i tapped: "  + tappedPointer)
                currentTappedPointerIndexBPM = (relativeTap * currentBPMList.size).toInt() //sublist Index
                tappedPointerIndexBPM =  currentTappedPointerIndexBPM + fromIndexBPM // original List Index

                println("INDEX IN ARRAY: " + tappedPointerIndexBPM)
                lastTappedPointerIndexBPM = tappedPointerIndexBPM
                zoomValue = if (zoomValue <= 30) 15 else zoomValue - 30
                currentCounter++



            })
```

**Code 14:** Einfaches Beispiel eines Sourcecode-Generators, abgeleitet von **??** und eigenen Ergänzungen

## 4.8 BPM

This code implements the Pan-Tompkins algorithm for detecting QRS complexes in an ECG signal. The algorithm processes the raw ECG data through multiple stages of filtering, differentiation, squaring, integration, and peak detection to identify heartbeats accurately.

The panTompkins function serves as the main processing pipeline. It first applies a bandpass filter, which is a combination of a low-pass and a high-pass filter, to remove noise and unwanted frequency components from the raw ECG data. The filtered signal then undergoes differentiation, enhancing the slope of the QRS complex.

Next, the squared transformation is applied to amplify higher amplitude variations, making QRS peaks more distinguishable. The resulting signal is further processed using moving window integration, which smooths the waveform and highlights the QRS complexes by averaging values over a short period.

Finally, peak detection is performed. A dynamic threshold is determined based on the signal's overall amplitude, and only peaks that exceed this threshold and maintain a minimum interval (to prevent false detections) are identified as QRS complexes.

The lowPassFilter and highPassFilter functions implement basic digital filters to isolate the frequency range of interest, while bandpassFilter applies them sequentially. The differentiate function enhances rapid signal changes characteristic of QRS complexes. The movingWindowIntegration function smooths the processed signal by averaging values over a predefined window.

The detectPeaks function then scans the processed data for local maxima that exceed a threshold and are sufficiently spaced apart (at least 200 milliseconds), ensuring only valid QRS peaks are detected. The algorithm ultimately returns a list of QRS peak positions, which can be used for heart rate analysis and further ECG interpretation.

```
1         fun panTompkins(ecgData: List<Int>): List<Int> {
2
3             val filteredData = bandpassFilter(ecgData)
4             val derivativeData = differentiate(filteredData)
5             val squaredData = derivativeData.map { it.toDouble().pow(2).
                  toInt() }
6             val integratedData = movingWindowIntegration(squaredData)
```

```
7           return detectPeaks(integratedData, SAMPLE_RATE)
8       }
9       fun lowPassFilter(signal: List<Int>): List<Int> {
10          var y = MutableList(signal.size) { 0 }
11          for (n in signal.indices) {
12              if (n >= 12) {
13                  y[n] = 2 * y[n - 1] - y[n - 2] + signal[n] - 2 * signal
                        [n - 6] + signal[n - 12]
14              } else {
15                  0
16              }
17          }
18          return y
19      }
20
21      fun highPassFilter(signal: List<Int>): List<Int> {
22          var y = MutableList(signal.size) { 0 }
23          for (n in signal.indices) {
24              if (n >= 32) {
25                  y[n] = y[n - 1] - (signal[n] / 32) + signal[n - 16] -
                        signal[n - 17] + (signal[n - 32] / 32)
26              } else {
27                  0
28              }
29          }
30          return y
31      }
32
33
34      fun bandpassFilter(ecgData: List<Int>): List<Int> {
35          var lp_filtered = lowPassFilter(ecgData)
36          var hp_filtered = highPassFilter(lp_filtered)
37          return hp_filtered
38      }
39
40
41      fun differentiate(data: List<Int>): List<Int> {
42          var y = MutableList(data.size) { 0 }
43          for (n in data.indices){
44              if (n >= 4) {
45                  y[n] = (2 * data[n] + data[n - 1] - data[n - 3] - 2 *
                        data[n - 4]) / 8
46              }
47              else{
48                  y[n] = 0
49              }
```

```
50
51              }
52
53              return y
54
55          }
56
57
58          fun movingWindowIntegration(data: List<Int>): List<Int> {
59              val result = mutableListOf<Int>()
60              for (i in data.indices) {
61                  val start = maxOf(0, i - WINDOW_SIZE / 2)
62                  val end = minOf(data.size, i + WINDOW_SIZE / 2)
63                  val average = data.subList(start, end).average()
64                  result.add(average.toInt())
65              }
66              return result
67          }
68
69
70          fun detectPeaks(signal: List<Int>, fs: Int): List<Int> {
71              val threshold = signal.sum() / signal.size
72              val peaks = mutableListOf<Int>()
73              val rrInterval = (0.2 * fs).toInt()
74              var lastPeak = -rrInterval
75
76              for (n in 1 until signal.size - 1) {
77                  if (signal[n] > threshold && signal[n] > signal[n - 1] &&
                        signal[n] > signal[n + 1]) {
78                      if (n - lastPeak >= rrInterval) {
79                          peaks.add(n)
80                          lastPeak = n
81                      }
82                  }
83              }
84
85              return peaks
86          }
```

**Code 15:** Einfaches Beispiel eines Sourcecode-Generators, abgeleitet von **??** und eigenen Ergänzungen

## 4.9 StorageHandler

This function provides a file selection mechanism using Jetpack Compose and Android's Storage Access Framework, allowing users to pick multiple files from their device. It utilizes rememberLauncherForActivityResult to open a file picker and handle the selection result.

When a user clicks the "Select doc" button, the launcher is triggered, allowing them to choose files. The selected file URIs are processed, and based on their type, they are categorized into a mutable map:

Binary files (.bin) are read using readShort(file _) and stored under the "bin" key. Metadata files (if the file contains "start") are split by ";" and stored under "meta". Other text files (e.g., BPM data) are also split by ";" and stored under "bpm". This function returns a map containing the parsed data, which can later be used for further processing, such as visualizing ECG and BPM data.

```
fun PickDocument(): MutableMap<String, Any>{

...


contract= ActivityResultContracts.OpenMultipleDocuments()){

    result.value = it
}
Column{
    Button(onClick = {
        launcher.launch(arrayOf("*/*"))
    }){
        Text(text="Select doc")
    }
    result.value?.forEach(){uri ->

        uri?.let{
            val file = saveUriToFile(context, it)
            if(uri?.let { checkFileType(context, it) } != null &&
                checkFileType(context, uri) == "application/octet-stream" ){
                dataMap["bin"] = file?.let { file_ -> readShort(file_) }!!
            }else{
                if (file != null && file.canRead() && file.readText().
                    contains("start")) {
                    dataMap["meta"] = file.readText().split(";")
                }else{
                    if (file != null) {
```

```
27                          dataMap["bpm"] = file.readText().split(";")}
28                      }
29                  }
30              }
31
32          }
33
34
35      }
36  }
37  return dataMap
38      }
39  }
```

**Code 16:** Einfaches Beispiel eines Sourcecode-Generators, abgeleitet von **??** und eigenen Ergänzungen

# 5 Summary

This thesis presents the design and implementation of a frontend application for real-time and long-term monitoring of electrocardiographic (ECG) data, utilizing Bluetooth Low Energy (BLE) communication. The primary objective was to develop an efficient and responsive system capable of handling continuous ECG data streams while ensuring smooth visualization and analysis. Key challenges included managing high-frequency data acquisition, real-time signal processing, and rendering an intuitive graphical representation. By leveraging Kotlin and Jetpack Compose, an optimized architecture was established, incorporating coroutines for concurrent data processing and an adaptive buffer for efficient data storage and retrieval.

# 6 Result

The developed system successfully processes and visualizes ECG data in real time. Through the implementation of a ring buffer and sliding window approach, the application maintains a continuous and smooth representation of the ECG signal, preserving critical waveform characteristics. The use of the Pan-Tompkins algorithm allows for reliable QRS peak detection, enhancing the accuracy of heartbeat identification. Additionally, BLE communication was optimized to minimize latency and packet loss, ensuring stable data transmission. The performance evaluation indicates that the system meets the requirements for real-time ECG monitoring, achieving both responsiveness and scalability.

# 7 Outlook

Future enhancements could focus on integrating machine learning techniques for advanced arrhythmia detection, further improving the diagnostic capabilities of the application. Additionally, expanding compatibility with multiple BLE-enabled ECG devices could increase the system's versatility. Implementing cloud synchronization for remote monitoring and historical data analysis would further extend the practical applications of this work. Lastly, optimizing power consumption for mobile usage remains a crucial aspect for improving long-term monitoring solutions.

# 8 Appendix

## 8.1 Versions

This section presents fundamental versions of the required development resources used for this work.

The tests for this application during the development process, were done on a phone using the **Android version 13** which implies the usage of **API level 33**. Utilizing lower Android version than 12 (API Level 31,32) is not possible due to new runtime permissions added in Android 13, making it impossible to start the app[5]. Higher Android versions were not reviewed. More versions of the used ressources, are presented in table 3.

**Table 3:** Versions of used resources

| **Ressource** | **Version used** |
|---|---|
| Android version | 13 |
| API Level | 33 |
| Koltin compiler extension version | 5.15.1 |
| minSDK | 30 |

## 8.2 Beispielcode zur Erzeugung eines Quelltext-Generators

**Code 17:** Einfaches Beispiel eines Sourcecode-Generators, abgeleitet von **??** und eigenen Ergänzungen

[5] https://developer.android.com/develop/connectivity/bluetooth/bt-permissions

## Literature references

S.J. Clarke. Selecting the correct tools and methods for software systems development. In *IEE Colloquium on Are Software Development Technologies Delivering Their Promise?*, page 7/1, 1995. doi: 10.1049/ic:19950387.

F. John Dian, Amirhossein Yousefi, and Sungjoon Lim. A practical study on bluetooth low energy (ble) throughput. In *2018 IEEE 9th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON)*, page 768, 2018. doi: 10.1109/IEMCON.2018.8614763.

Mandahadi Kusuma, Ahmad Haris Rifani, and Bambang Sugiantoro. Comparison analysis of jetpack compose and flutter in android-based application development using technical domain. In *2023 Eighth International Conference on Informatics and Computing (ICIC)*, pages 3–5, 2023. doi: 10.1109/ICIC60109.2023.10381987.